

ELEC473 Assignment 4 - Synthesising the NIOS II Processor

Ryan McKee - 201954749

August 5, 2026

Abstract

This report documents the design, synthesis and testing of a Nios II soft-core system on the Altera/Terasic DE2 (Cyclone II) development board using Quartus Prime and Qsys (Platform Designer). Part A builds a base Nios II system with on-chip memory, a JTAG UART, an interval timer, parallel I/O and a system ID peripheral, and extends it with the board's 512 KB SRAM and 8 MB SDRAM, verified using the Nios II IDE memory test utility. Part B extends this system with a custom combinational instruction, `count_leading_ones`, that counts the number of leading '1' bits in a 32-bit word in a single cycle. Its design is presented as a Mealy finite state machine, implemented in Verilog, and integrated into the system as a Nios II custom instruction slave. Its performance is measured against an equivalent C software routine, with the hardware instruction found to be approximately $8\times$ faster on average.

Contents

1	Introduction	2
2	Part A - SRAM & SDRAM	4
2.1	System Configuration	4
2.1.1	Memory Testing Locations	7
2.1.2	SRAM Test	8
2.1.3	SDRAM Test	9
3	Part B - Custom Instruction	12
3.1	Algorithm Design	12
3.2	Software Implementation	13
3.2.1	Software Performance Result	15
3.3	Hardware Custom Instruction	16
3.3.1	Hardware Performance Result	18
3.4	Comparison	18
4	Part C - PWM Module	20
4.1	Design	20
4.2	PWM Peripheral	22
4.3	PWM Level Custom Instruction	24
4.4	System Integration and Pin Mapping	27
4.5	Software Design	28
4.6	Testing and Results	29
5	Conclusions	31
5.1	What Worked	31
5.2	What Didn't Work Initially	31
5.3	Critical Assessment	32
5.4	How the System Was Fully Tested	32

Chapter 1

Introduction

This assignment builds familiarity with FPGA (Field Programmable Gate Array) engineering through the use of **Qsys** [1] and the **Nios II** [3] processor. Nios II is **Altera's** [2] own 32-bit embedded processor micro-architecture, designed specifically for synthesis on FPGA. Nios II is the successor to Altera's first configurable 16-bit embedded processor, Nios, introduced in 2000. Throughout this assignment we target the Cyclone II FPGA on the Altera DE2 board, which we synthesise designs for, bench test and program over the course of the work.

Nios II offers the following design parameters:

- 32 general-purpose 32-bit registers.
- A full 32-bit instruction set, data path, and address space.
- A single-instruction 32×32 multiply and divide, producing a 32-bit result.

Nios II's soft-core nature (a soft microprocessor is a microprocessor core wholly implemented using logic synthesis, and can be targeted at different semiconductor devices containing programmable logic, e.g. FPGA or CPLD, spanning both high-end and commodity applications [4]) enables hardware engineers to specify and generate a custom Nios II core, tailored to their own application requirements.

Qsys (now known as Platform Designer) is a system integration tool for Intel Quartus Prime. It connects multiple intellectual property (IP) blocks, processors (such as Nios II), and memory interfaces using standard interconnects such as Avalon, and can be used to generate the resulting system as synthesisable HDL.

This assignment is split into three parts. **Part A** synthesises a Nios II processor system with 20 KB of on-chip memory, a timer, a JTAG UART, 8 parallel I/O pins and a system ID component, then interfaces the DE2 board's 512 KB SRAM and 8 MB SDRAM to this design. The design is tested using the memory test programs within the Nios II IDE, printing name and student ID to the terminal each time a

memory test is run. **Part B** continues from Part A, adding a custom instruction that counts the number of leading ‘1’ bits in a 32-bit word, together with a C test program that exercises the instruction. A software routine implementing the same functionality is also written and benchmarked in C, so that the custom instruction’s execution speed can be compared against an equivalent software implementation. **Part C** uses PWM to control the intensity of LEDR10.

Chapter 2

Part A - SRAM & SDRAM

2.1 System Configuration

The base system is built in Qsys/Platform Designer around the Nios II processor, with 20 KB of on-chip memory, a JTAG UART for the console, an interval timer, an 8-bit parallel I/O peripheral (`led_pio`), and a System ID peripheral, following a top-down design flow: the processor and its minimal support peripherals were brought up and verified first, and the SRAM, SDRAM, and (in later parts) the custom hardware were each added and re-verified as separate, self-contained increments on top of that working base. The board's SRAM and SDRAM controllers are added and wired onto the same Avalon Memory-Mapped interconnect as separate memory-mapped slaves, addressable from the Nios II data master. Figure 2.1 shows the resulting top-level Quartus block diagram (BDF), instantiating the generated Qsys system against the DE2 board's physical SRAM, SDRAM and LED pins.

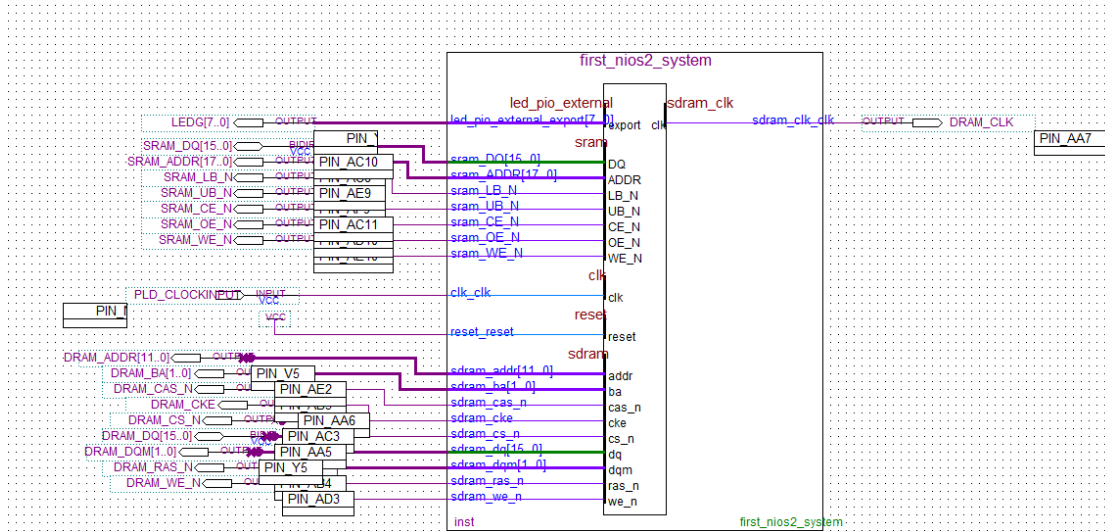


Figure 2.1: Top-level Quartus block diagram (.bdf) for the Part A system. The Qsys-generated `first_nios2_system` symbol is instantiated once and wired directly to the board-level pins: the SRAM data/address/control bus (`SRAM_*`), the SDRAM bus (`DRAM_*`) together with its dedicated `sdrank_clk` output pin, and the 8-bit LEDG parallel I/O export.

The board’s default 50 MHz oscillator is distributed to the system through a `clocks` (DE-series board peripheral clock) component, which also derives a second, phase-advanced `sdrank_clk` output (visible as a separate Clock Output in Figure 2.2) that drives the external `DRAM_CLK` pin ahead of the internal system clock, following the assignment brief’s guidance to advance the SDRAM clock by approximately 3 ns relative to the system clock using a PLL. This phase advance compensates for the round-trip propagation delay to and from the external SDRAM chip, so that the address, data and command signals are correctly set up and held at the SDRAM device’s own clock edge rather than the FPGA’s internal edge.

Figure 2.2 shows the full Qsys/Platform Designer system contents for the completed design, including the base addresses assigned to each component by the Qsys address map. This single system is shared across all three parts of the assignment: the SRAM and SDRAM controllers from Part A are visible alongside the `count_leading_ones` custom instruction slave added in Part B (opcode 0) and the `pwm` peripheral and `nci_pwm_controller` custom instruction added in Part C (opcode 1), described in the following chapters.

Use	Connections	Name	Description	Export	Clock	Base	End	RQ	Opcode Name
✓		On-Chip Memory (RAM or ROM)							
✓		On-Chip Memory (RAM or ROM)							
✓		clk1	Clock Input	Double-click to export	clocks_sys...	0x0008_0000	0x0008_ffff		
✓		s1	Avalon Memory Mapped Slave	Double-click to export	[clk1]				
✓		reset1	Reset Input	Double-click to export	[clk1]				
✓		cpu	Nios II Processor						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset_n	Reset Input	Double-click to export	[clk]				
✓		data_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
✓		instruction_master	Avalon Memory Mapped Master	Double-click to export	[clk]				
✓		jtag_debug_module_reset	Reset Output	Double-click to export	[clk]				
✓		jtag_debug_module	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0008_8000	0x0008_ffff		
✓		custom_instruction_master	Custom Instruction Master	Double-click to export					
✓		JTAG UART	JTAG UART						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clk]				
✓		avalon_jtag_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0008_9038	0x0008_903f		
✓		sys_clk_timer	Interval Timer						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clk]				
✓		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0008_9000	0x0008_901f		
✓		sysid	System ID Peripheral						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clk]				
✓		control_slave	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0008_9030	0x0008_9037		
✓		led_pio	PIO (Parallel I/O)						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clk]				
✓		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0008_9020	0x0008_902f		
✓		external_connection	Conduit	led_pio_external					
✓		sram	SRAM/SSRAM Controller						
✓		clock_reset	Clock Input	Double-click to export	clocks_sys...				
✓		clock_reset_reset	Reset Input	Double-click to export	[clock_reset]				
✓		external_interface	Conduit	sram					
✓		avalon_sram_slave	Avalon Memory Mapped Slave	Double-click to export	[clock_reset]	0x0000_0000	0x0007_ffff		
✓		clocks	Clock Signals for DE-series Board Per...						
✓		clk_n_primary	Clock Input	reset	exported				
✓		clk_n_primary_reset	Reset Input	Double-click to export	[clk_n_prima...				
✓		sys_clk	Clock Output	Double-click to export	clocks_sys...				
✓		sys_clk_reset	Reset Output	Double-click to export	clocks_sys...				
✓		sdram_clk	Clock Output	Double-click to export	clocks_sdra...				
✓		sdram	SDRAM Controller						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clk]				
✓		s1	Avalon Memory Mapped Slave	Double-click to export	[clk]	0x0100_0000	0x017f_ffff		
✓		wire	Conduit	sdram					
✓		count_leading_ones	count_leading_ones						
✓		nios_custom_instruction_slave_0	Custom Instruction Slave	Double-click to export		Opcode 0	Opcode 0		count_leading_ones_0
✓		pwm	pwm						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		avalon_slave_0	Avalon Memory Mapped Slave	Double-click to export	[clock]	0x0008_8000	0x0008_8003		
✓		conduit_end	Conduit	pwm_out					
✓		reset_n	Reset Input	Double-click to export	[clock]				
✓		nci_pwm_controller_0	nci_pwm_controller						
✓		clk	Clock Input	Double-click to export	clocks_sys...				
✓		reset	Reset Input	Double-click to export	[clock]				
✓		nios_custom_instruction_slave_0	Custom Instruction Slave	Double-click to export		Opcode 1	Opcode 1		nci_pwm_controller_0
✓		avalon_master_0	Avalon Memory Mapped Master	Double-click to export					

Figure 2.2: Qsys/Platform Designer system contents for the completed system, showing the Nios II processor, on-chip memory, JTAG UART, interval timer, System ID, parallel I/O, SRAM and SDRAM controllers (Part A), the `count_leading_ones` custom instruction slave (Part B, opcode 0), and the `pwm` peripheral with its `nci_pwm_controller` custom instruction (Part C, opcode 1), together with their assigned Avalon base/end addresses.

Table 2.1 transcribes the Avalon Memory-Mapped address map from Figure 2.2 into a full system memory map, and Table 2.2 lists the two custom instructions separately, since they are addressed by opcode rather than by memory address.

Table 2.1: Complete Avalon Memory-Mapped peripheral address map for the final system (Parts A–C), as generated by Qsys/Platform Designer.

Component	Base Address	End Address
SRAM (<code>sram.avalon_sram_slave</code>)	0x0000_0000	0x0007_FFFF
On-chip memory (<code>onchip_mem</code> , 20 KB)	0x0008_0000	0x0008_4FFF
PWM peripheral (<code>pwm.avalon_slave_0</code>)	0x0008_8000	0x0008_8003
CPU JTAG debug module	0x0008_8800	0x0008_8FFF
Interval timer (<code>sys_clk_timer</code>)	0x0008_9000	0x0008_901F
LED parallel I/O (<code>led_pio</code>)	0x0008_9020	0x0008_902F
System ID (<code>sysid</code>)	0x0008_9030	0x0008_9037
JTAG UART (<code>jtag_uart</code>)	0x0008_9038	0x0008_903F
SDRAM (<code>sdram.s1</code>)	0x0100_0000	0x017F_FFFF

Table 2.2: Custom instruction opcode assignments (opcode-addressed, not memory-mapped).

Custom instruction	Opcode	Type
<code>count_leading_ones_0</code> (Part B)	0	Combinational
<code>nci_pwm_controller_0</code> (Part C)	1	Extended

To exercise both memories, the Nios II IDE memory test example program was flashed and run twice: once with the test program itself (`.text`, `.rodata`, `.rwddata`, heap and stack) residing in SRAM while SDRAM was tested, and once with the program residing in SDRAM while SRAM was tested. This is necessary because the memory test is destructive to the region under test, so the code and data running the test can never overlap the region being tested, otherwise the test would corrupt itself and hang or fail.

2.1.1 Memory Testing Locations

Table 2.3 lists the address regions tested for each memory device, taken from the Qsys-assigned base addresses shown in Figure 2.2.

Table 2.3: Memory testing address regions			
Region	Origin	Length	End Address
SRAM	0x00000000	0x80000 (512 KB)	0x0007FFFF
SDRAM	0x01000000	0x800000 (8 MB)	0x017FFFFF

2.1.2 SRAM Test

Because the memory test is destructive, the executable running the test could not itself reside in the region under test. This was controlled at the Qsys/Platform Designer level by setting the Nios II Processor's *Reset Vector* and *Exception Vector* memory (Figure 2.5): to test SRAM, the reset/exception vector memory was set to `sdram.s1`, so the whole program (`.text`, `.rodata`, `.rdata`, heap and stack) was linked into SDRAM by the BSP's default (auto) linker section placement, leaving the full 512 KB SRAM region free for the destructive test. Figure 2.3 shows the resulting test, run over the full SRAM range from `0x00000000` to `0x007FFFFF`, with the modified example printing the author's name and student ID on start-up as required by the assignment brief.

```

<---->  Nios II Memory Test.  <---->
Name: Ryan McKee   ID: 201954749
This software example tests the memory in your system to assure it
is working properly.  This test is destructive to the contents of
the memory it tests.  Assume the memory being tested does not contain
the executable or data sections of this code or the exception address
of the system.

-----
Memory Test Main Menu
-----
a:  Test RAM
b:  Test Flash
q:  Exit
-----

Select Choice (a-b): [Followed by <enter>]
-ERROR:
is an invalid entry.  Please try again

Press enter to continue...

-----
Memory Test Main Menu
-----
a:  Test RAM
b:  Test Flash
q:  Exit
-----

Select Choice (a-b): [Followed by <enter>]
A
Base address to start memory test: (i.e. 0x8000000)
>
0x00000000
End Address:
>
0x007FFFFF

Name: Ryan McKee   ID: 201954749
Testing RAM from 0x0 to 0x7FFFFF
-Data bus test passed
-Address bus test passed
-Byte and half-word access test passed
-Testing each bit in memory device. . .  passed
Memory at 0x0 Okay

Press enter to continue...

```

Figure 2.3: Nios II IDE Memory Test utility testing the full 512 KB SRAM region (0x00000000–0x007FFFFF), with the test program itself executing from SDRAM. All sub-tests (data bus, address bus, byte/half-word access, and per-bit) pass.

2.1.3 SDRAM Test

To test SDRAM, the same mechanism was used in reverse: the Nios II Processor’s reset/exception vector memory was set to `sram.avalon_sram_slave` (Figure 2.4),

so the program built entirely into SRAM, leaving SDRAM entirely free for the destructive test. Without this step in either direction, the running program's own code and data would have been overwritten mid-test, causing the test to hang or corrupt itself.

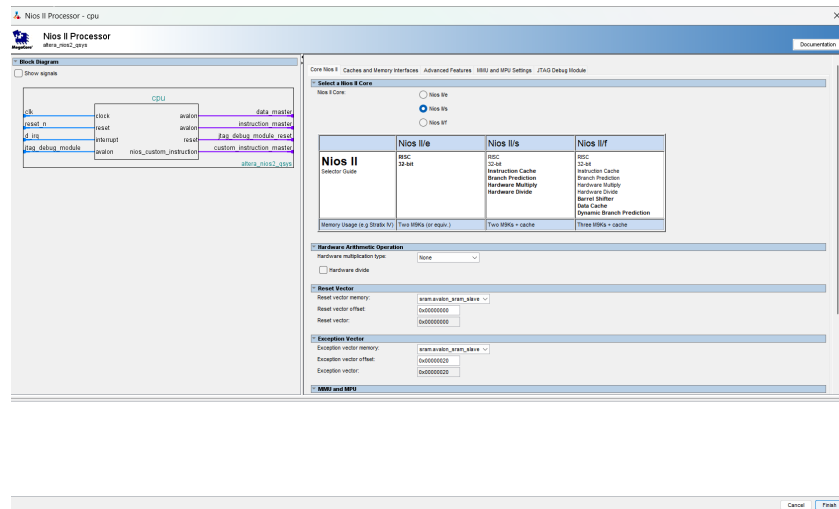


Figure 2.4: Nios II Processor component configuration used when testing SDRAM: the reset and exception vector memory is set to `sram.avalon_sram_slave` (offset 0x00000000), placing the whole program in SRAM.

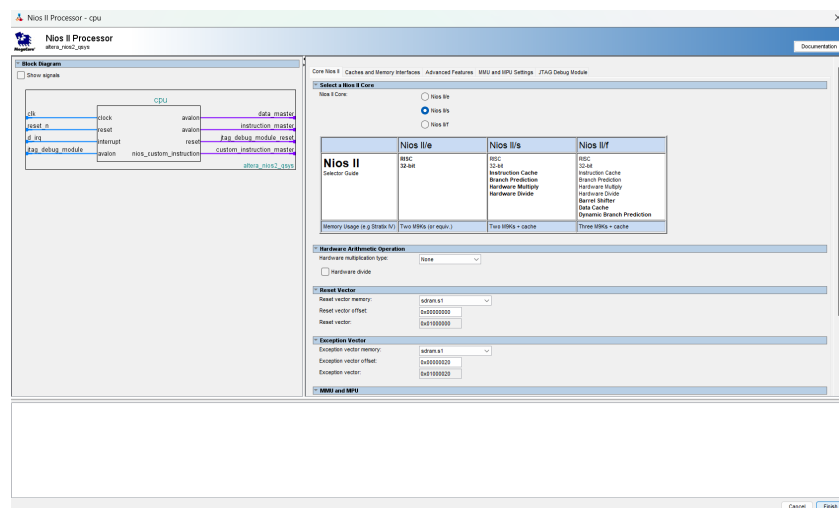


Figure 2.5: Nios II Processor component configuration used when testing SRAM: the reset and exception vector memory is set to `sdram.s1` (offset 0x01000000), placing the whole program in SDRAM.

```

<---->  Nios II Memory Test.  <---->
Name: Ryan McKee
Student ID: 201954749
This software example tests the memory in your system to assure it
is working properly. This test is destructive to the contents of
the memory it tests. Assure the memory being tested does not contain
the executable or data sections of this code or the exception address
of the system.

-----
Memory Test Main Menu
-----
a: Test RAM
b: Test Flash
q: Exit
-----

Select Choice (a-b): [Followed by <enter>]
a
Base address to start memory test: (i.e. 0x800000)
>|
0x1000000
End Address:
>
0x17FFFFFF

Testing RAM from 0x1000000 to 0x17FFFFFF
-Data bus test passed
-Address bus test passed
-Byte and half-word access test passed
-Testing each bit in memory device. . . passed
Memory at 0x1000000 Okay

Press enter to continue...

```

Figure 2.6: Nios II IDE Memory Test utility testing the full 8 MB SDRAM region (0x1000000–0x17FFFFFF), with the test program itself executing from SRAM. All sub-tests pass, and the memory reports Okay.

Chapter 3

Part B - Custom Instruction

3.1 Algorithm Design

The goal of the custom instruction is to count the number of leading ‘1’ bits in a 32-bit word, i.e. the length of the run of ‘1’s starting at the most significant bit (MSB) and ending at the first ‘0’ bit (or at the LSB, if the word is all ones). Before implementing this in Verilog, the algorithm was designed as a Mealy finite state machine (Harris & Harris style), treating the 32-bit input word as a serial bit-stream presented MSB-first.

The machine has two states: S_1 , the *counting* state, and S_2 , the *done* state. Starting in S_1 , every ‘1’ bit received increments a running counter and keeps the machine in S_1 ; the first ‘0’ bit received transitions the machine to S_2 without incrementing the counter, and the machine then remains in S_2 (absorbing any remaining bits with no further effect) since the leading-ones run is over. Figure 3.1 shows the resulting state diagram.

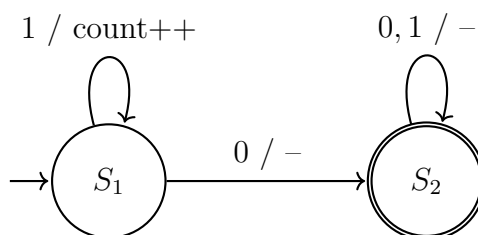


Figure 3.1: Mealy machine for the leading-ones counting algorithm. The input word is streamed MSB $\rightarrow$ LSB. S_1 (initial state) counts consecutive leading ‘1’ bits; the first ‘0’ bit moves the machine to the terminal state S_2 , after which the count no longer changes.

Formally, the machine is the 6-tuple $M = (Q, \Sigma, \Gamma, \delta, \omega, q_0)$, where:

- $Q = \{S_1, S_2\}$ – the finite set of states.
- $\Sigma = \{0, 1\}$ – the finite input alphabet (one bit per clock/step).
- $\Gamma = \mathbb{Z}^+$ – the output alphabet (the running leading-ones count).
- $q_0 = S_1$ – the initial state.
- $\delta : Q \times \Sigma \rightarrow Q$ – the state transition function, given in Table 3.1.
- $\omega : Q \times \Sigma \rightarrow \Gamma$ – the Mealy output function: emits `count++` on $\delta(S_1, 1)$, and no output (count held) otherwise.

Table 3.1: State transition function δ

Current state	Input	Next state	Output ω
S_1	1	S_1	<code>count++</code>
S_1	0	S_2	–
S_2	0	S_2	–
S_2	1	S_2	–

Because the FSM only ever needs to hold a running count and knowledge of whether a ‘0’ bit has already been seen, it maps directly onto a combinational priority-encoder style circuit rather than a literal bit-serial shift register: the same result can be produced in a single clock cycle by evaluating the run of leading ones over the whole 32-bit word at once, halving the search space at each stage (16/8/4/2/1 bits). This is the structure used for the Verilog implementation in Section 3.3.

3.2 Software Implementation

Before implementing the custom hardware instruction, a reference software routine implementing the same leading-ones-count algorithm was written in C for the Nios II IDE, so that its speed could later be compared against the hardware instruction. `count_leading_ones()` implements the bit-serial algorithm from Figure 3.1 directly: it walks the word from bit 31 down to bit 0, incrementing a counter for each ‘1’ bit and breaking on the first ‘0’. A `PERFORMANCE.TEST` mode is included, which repeats the count over 100,000 pseudo-varied input words and reports the average execution time per call using the standard C `clock()` API.

```

#include <stdio.h>
#include <stdint.h>
/* #define PERFORMANCE_TEST */
#define PERFORMANCE_TEST
#ifdef PERFORMANCE_TEST
#include <time.h>
#define PERFORMANCE_TEST_ITERATIONS 100000u
#endif
static const uint32_t word = 0xFA000000;
int count_leading_ones(uint32_t value)
{
    uint32_t leading_ones = 0;
    int i;
    for (i = 31; i >= 0; i--)
    {
        uint32_t bit = (value >> i) & 1;
        if (!bit) {
            break;
        }
        ++leading_ones;
    }
    return leading_ones;
}
#ifdef PERFORMANCE_TEST
static void run_performance_test(void)
{
    clock_t start, end;
    clock_t elapsed_ticks;
    uint32_t elapsed_us;    /* total time in microseconds, integer */
    uint32_t avg_x100_us;   /* average per call, in hundredths of a
        microsecond */
    uint32_t i;
    volatile int result = 0; /* volatile so the compiler can't optimize the
        loop away */

    printf("Running performance test (%u iterations)...\n",
        PERFORMANCE_TEST_ITERATIONS);

    start = clock();
    for (i = 0; i < PERFORMANCE_TEST_ITERATIONS; i++)
    {
        /* Vary the input word each iteration so the result can't just
         * be computed once and cached by the compiler. */
        uint32_t test_word = word ^ i;
        result = count_leading_ones(test_word);
    }
    end = clock();

```



```

    elapsed_ticks = end - start;
    /* Convert clock ticks to microseconds using integer math only
     * (avoids the small-C-library printf %f limitation). */
    elapsed_us = (uint32_t)(((uint64_t) elapsed_ticks * 1000000u) /
        CLOCKS_PER_SEC);
    /* Average per call, scaled by 100 so we can print two decimal places
     * without any floating point. */
    avg_x100_us = (uint32_t)(((uint64_t) elapsed_us * 100u) /
        PERFORMANCE_TEST_ITERATIONS);

    printf("Performance test complete.\n");
    printf(" Total time  : %lu us (%lu ticks)\n",
        (unsigned long) elapsed_us, (unsigned long) elapsed_ticks);
    printf(" Avg per call : %lu.%02lu us\n",
        (unsigned long) (avg_x100_us / 100u),
        (unsigned long) (avg_x100_us % 100u));
    printf(" Last result  : %d\n", result);

    if (elapsed_ticks < 10)
    {
        printf(" Warning: elapsed ticks very low (%lu) - increase\n"
            "   PERFORMANCE_TEST_ITERATIONS for a more reliable\n"
            "   measurement; clock() resolution is limited by the\n"
            "   system timer period.\n", (unsigned long) elapsed_ticks);
    }
}
#endif
int main()
{
    printf("Hello from Nios II!\n");
#ifdef PERFORMANCE_TEST
    run_performance_test();
#else
    {
        uint32_t leading_ones = count_leading_ones(word);
        printf("leading ones %lu\n", (unsigned long) leading_ones);
    }
#endif
    return 0;
}

```

3.2.1 Software Performance Result

Running the software routine for 100,000 iterations on the Nios II system gives an average execution time of 24.63 μ s per call:

```

Running performance test (100000 iterations)...
Performance test complete.

```

```
Total time : 2463000 us (2463 ticks)
Avg per call : 24.63 us
Last result : 5
```

Because the loop breaks as soon as the first ‘0’ bit is encountered, this cost is data-dependent: words with fewer leading ones return faster, while a word of all ‘1’ bits takes the full 32 iterations.

3.3 Hardware Custom Instruction

The FSM from Figure 3.1 was re-expressed as a combinational priority-encoder tree (rather than a literal bit-serial state machine) so that it could be implemented as a Nios II *combinational* custom instruction – one that produces its result within the same clock cycle it is issued, with no internal state or extra latency. The word is repeatedly halved: first checking whether the upper 16 bits are all ‘1’, then the upper 8 of whichever half was selected, and so on down to a single bit, accumulating the count as each stage resolves. This directly mirrors $\delta(S_1, 1)$ (stay in S_1 , increment count) versus $\delta(S_1, 0)$ (move to S_2 , stop counting) from the FSM, but evaluates all bit positions in parallel instead of serially.

```
module count_leading_ones (
    // Nios II custom instruction slave interface
    // "ncs_" prefix lets Platform Designer auto-recognize these as the
    // Nios II custom instruction slave interface (clk, clk_en, reset,
    // dataa, result) instead of guessing a generic Avalon-MM slave.
    ncs_clk,
    ncs_clk_en,
    ncs_reset,
    ncs_dataa,
    ncs_result
);

input      ncs_clk;
input      ncs_clk_en;
input      ncs_reset;
input [31:0] ncs_dataa;
output [31:0] ncs_result;

// Priority tree, halving each stage - pure combinational.

wire msb16_all1 = (ncs_dataa[31:16] == 16'hFFFF);
wire [15:0] next16 = msb16_all1 ? ncs_dataa[15:0] : ncs_dataa[31:16];

wire msb8_all1 = (next16[15:8] == 8'hFF);
wire [7:0] next8 = msb8_all1 ? next16[7:0] : next16[15:8];
```

```

wire msb4_all1 = (next8[7:4] == 4'hF);
wire [3:0] next4 = msb4_all1 ? next8[3:0] : next8[7:4];

wire msb2_all1 = (next4[3:2] == 2'b11);
wire [1:0] next2 = msb2_all1 ? next4[1:0] : next4[3:2];

wire msb1_all1 = next2[1];
wire bit0 = msb1_all1 ? next2[0] : 1'b0;

assign ncs_result = (msb16_all1 ? 16 : 0)
    + (msb8_all1 ? 8 : 0)
    + (msb4_all1 ? 4 : 0)
    + (msb2_all1 ? 2 : 0)
    + (msb1_all1 ? 1 : 0)
    + (msb1_all1 & bit0 ? 1 : 0);

endmodule

```

Once the Verilog above was synthesised, it was imported into Qsys as a Nios II custom instruction component (the `ncs_*` port prefix lets Platform Designer auto-detect it as a custom instruction slave rather than a generic Avalon-MM slave) and connected to the Nios II core's `custom_instruction_master` interface, as shown in Figure 3.2 (count_leading_ones_0, opcode 0). This exposes the circuit in software as the GCC built-in `__builtin_custom_ini(...)` custom instruction call, which the Nios II compiles down to a single custom instruction.

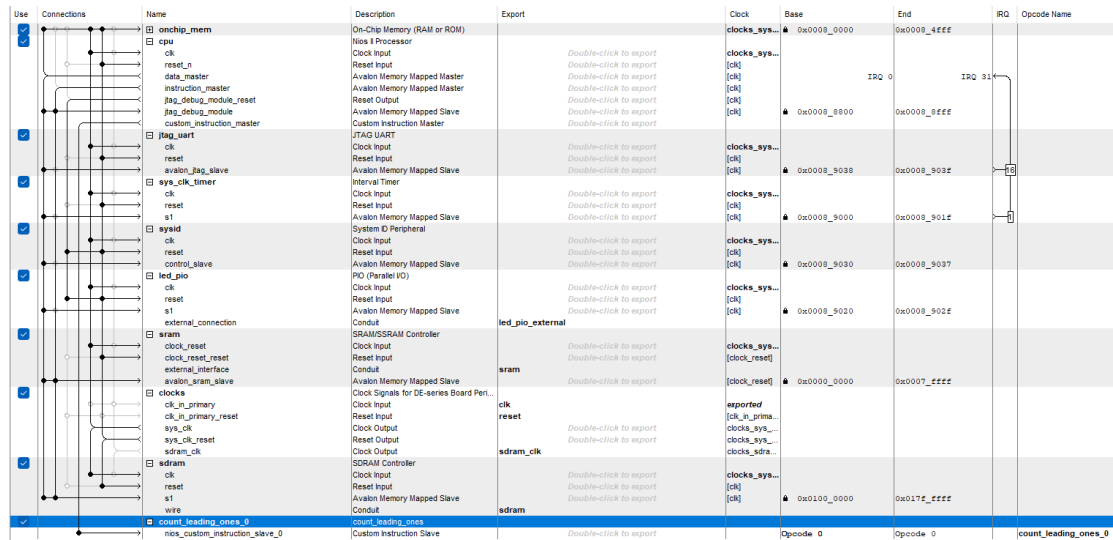


Figure 3.2: `count_leading_ones_0` connected as a Custom Instruction Slave on the Nios II core's `custom_instruction_master` interface, assigned opcode 0 by Platform Designer.

3.3.1 Hardware Performance Result

Running the equivalent test with the leading-ones count performed by the custom instruction instead of the C loop, for the same 100,000 iterations, gives an average execution time of 3.03 μ s per call:

```
Running performance test (100000 iterations)...
Performance test complete.
Total time : 303000 us (303 ticks)
Avg per call : 3.03 us
Last result : 5
```

Unlike the software loop, this cost is constant regardless of the input word, since the combinational priority-encoder tree resolves every bit position in the same clock cycle no matter how many leading ones are present.

3.4 Comparison

Table 3.2: Software vs. hardware leading-ones count, 100,000 iterations

	Software (C loop)	Hardware (custom instruction)
Total time	2463000 μ s	303000 μ s
Avg per call	24.63 μ s	3.03 μ s

The custom instruction is roughly $24.63/3.03 \approx 8.1\times$ faster on average than the software routine. This is the expected outcome: the software version executes a data-dependent loop of up to 32 iterations, each costing at least a shift, mask, compare, branch and increment, whereas the hardware version resolves the whole 32-bit priority-encoder tree combinationally in a single cycle, with the remaining measured time in both cases dominated by the fixed overhead of the function/instruction call, loop bookkeeping in the benchmark harness, and `clock()` itself.

Software routine - pros: no additional FPGA logic resources are consumed, it is portable to any Nios II configuration (or indeed any target) without hardware changes, and it is trivial to modify or debug. Cons: execution time is both slower and data-dependent (worst-case 32 iterations for an all-ones word), and it consumes instruction-fetch and data-path bandwidth on the processor for every iteration of the loop.

Hardware custom instruction - pros: constant, single-cycle latency regardless of input value, and it frees the processor from executing a software loop at all. Cons: it consumes FPGA logic resources (comparators and multiplexers for the priority-encoder tree), occupies one of a limited number of custom instruction

opcode slots, requires the Qsys system to be regenerated and the FPGA reprogrammed whenever it changes, and is tied to this specific Nios II system rather than being portable in software alone.

Overall, for a leading-ones count that is executed frequently or on a performance-critical path, the custom instruction is clearly the better choice, trading a small, fixed amount of FPGA fabric for an $\sim 8\times$ speed-up and deterministic timing. For infrequent or non-critical use, the software routine is preferable, since it avoids consuming custom instruction and logic resources that could otherwise be used elsewhere in the design.

Chapter 4

Part C - PWM Module

For Part C, PWM is used to gradually vary the intensity of LEDR10 from off to fully on and back to off, repeating continuously, as required by the assignment brief. Two hardware modules are used together: `pwm`, an Avalon-MM slave peripheral that generates the actual PWM waveform from an 8-bit duty register, and `ci_pwm_controller`, a Nios II custom instruction that lets software set the LED brightness using a simple 0-9 level rather than a raw 8-bit duty value.

4.1 Design

The assignment brief suggests two ways to implement the PWM generator: the simplest is to expose the duty register as a plain Qsys GPIO component and build the counter and comparator as free-standing logic outside Qsys, wiring the GPIO's parallel output into it by hand at the BDF level; the more elegant alternative is to package the register, counter and comparator together as a single custom Avalon-MM slave component inside Qsys. The GPIO route was rejected here because it splits one logical peripheral across two design domains - a Qsys-managed register and a hand-wired BDF block - doubling the places a clocking or reset mistake could be introduced, and because it exposes the raw duty register directly on the Avalon bus with no scope to add Avalon-native handshaking (e.g. the write-strobe re-timing described below) without leaving Qsys entirely. The single-component approach was chosen instead: it keeps the counter and comparator under the same reset domain and clock as the register that drives them, lets Qsys's own address/interrupt allocation manage the peripheral like any other memory-mapped device, and, as Section 3.3 shows, made it straightforward to later add a second, custom instruction front-end (`ci_pwm_controller`) onto the same underlying `pwm` block without touching its internal register/counter/comparator logic.

The core PWM design - a duty register, an up-counter, and a comparator be-

tween them, exactly the three-part structure suggested in the assignment brief - was originally developed as a standalone module in a previous assignment. For this assignment it was re-purposed as an Avalon-MM slave so that it could be instantiated and wired up entirely within Qsys/Platform Designer, rather than as a free-standing block connected by hand outside the generated system. Figure 4.1 shows the module's external interface and its internal register structure, and Figure 4.2 shows the ASM chart for its controller, covering reset, the write-strobe path that loads a new duty value, and the two symmetric counting paths that drive `pwm_out` high while the counter is below the duty value and low otherwise.

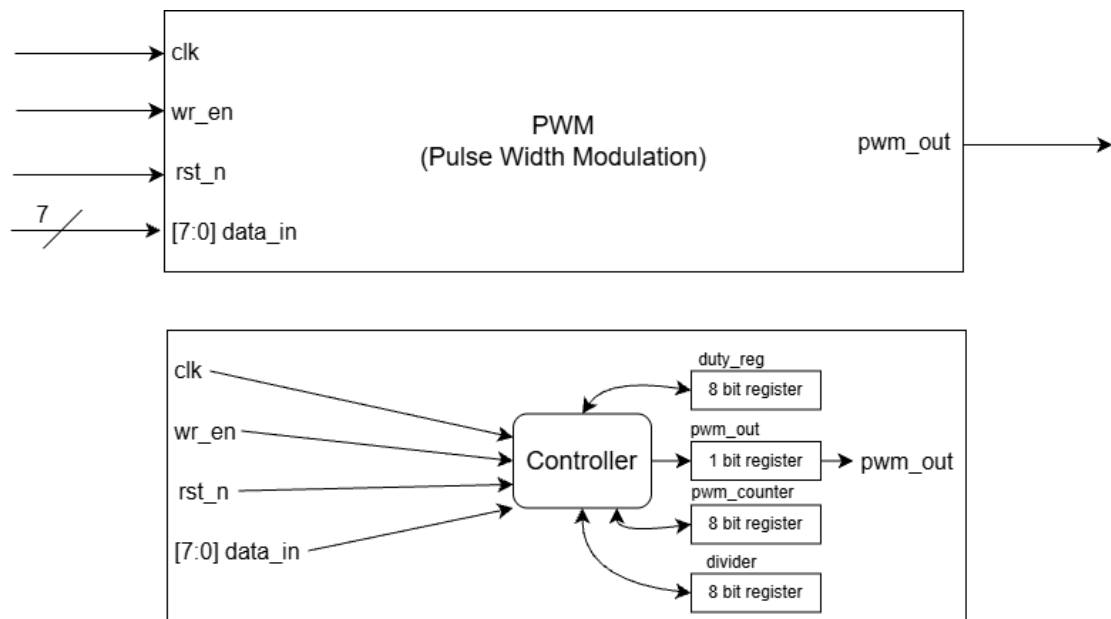


Figure 4.1: `pwm` module block diagram: external interface (top) and internal register structure - the duty register, PWM counter, clock divider, and registered output (bottom).

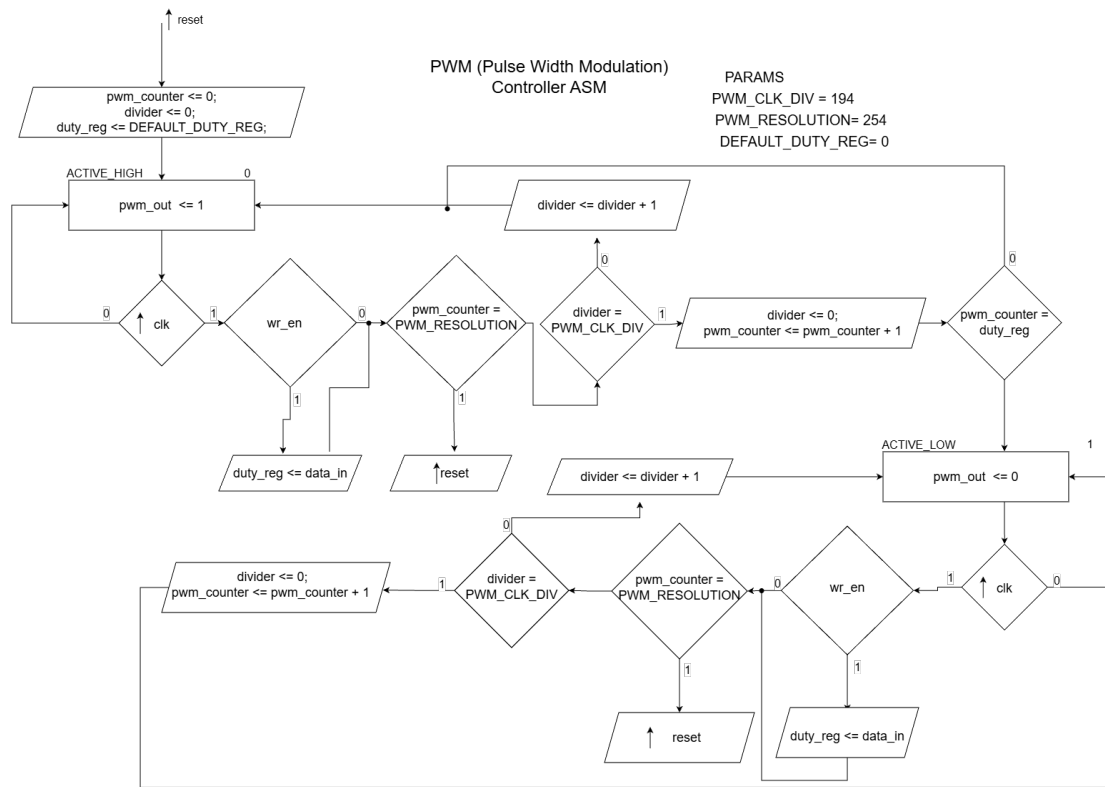


Figure 4.2: ASM chart for the `pwm` module’s controller, covering reset initialisation, the write-strobe path (`wr_en` loads `data_in` into `duty_reg`), and the symmetric high/low counting paths that compare `pwm_counter` against `duty_reg` to drive `pwm_out`.

4.2 PWM Peripheral

Making the module Avalon-MM compliant required more than just adding a register: the port names (`reset_n`, `write`, `writedata`) were chosen specifically to match Avalon-MM naming conventions, so that Qsys Component Editor’s “Analyze Synthesis Files” step auto-detects the clock, reset and Avalon slave interfaces without any manual signal-role assignment. The write path was also re-timed so that `pwm_out` registers from the same next-state values used to update `pwm_counter` and `duty_reg`, removing a delta-cycle race that would otherwise exist between the counter changing and `pwm_out` being sampled against it.

```
module pwm #(
    parameter PWM_CLK_DIV      = 194,
    parameter PWM_RESOLUTION   = 254,
    parameter [7:0] DEFAULT_DUTY_REG = 8'h02
```



```

)(
    input          clk,
    input          reset_n,
    input          write,
    input [31:0]   writedata,
    output reg     pwm_out
);
    reg [7:0] duty_reg;
    reg [7:0] pwm_counter;
    reg [7:0] divider;
    // Next-state values computed one cycle ahead, so pwm_out registers
    // from the SAME values used to update pwm_counter/duty_reg. This
    // removes the delta-cycle race with anything sampling pwm_out
    // right after pwm_counter changes.
    reg [7:0] next_counter;
    reg [7:0] next_duty;
    always @(*) begin
        next_duty = write ? writedata[7:0] : duty_reg;
        if (pwm_counter == PWM_RESOLUTION)
            next_counter = 8'd0;
        else if (divider == PWM_CLK_DIV)
            next_counter = pwm_counter + 1'b1;
        else
            next_counter = pwm_counter;
    end
    always @(posedge clk or negedge reset_n) begin
        if (!reset_n) begin
            pwm_counter <= 8'd0;
            divider <= 8'd0;
            duty_reg <= DEFAULT_DUTY_REG;
            pwm_out <= (8'd0 < DEFAULT_DUTY_REG);
        end else begin
            duty_reg <= next_duty;
            pwm_counter <= next_counter;
            if (pwm_counter == PWM_RESOLUTION) begin
            end else if (divider == PWM_CLK_DIV) begin
                divider <= 8'd0;
            end else begin
                divider <= divider + 1'b1;
            end
            pwm_out <= (next_counter < next_duty);
        end
    end
endmodule

```

4.3 PWM Level Custom Instruction

Driving `pwm` directly from software means every LED brightness change requires computing and writing an 8-bit duty value, and, since it is a plain Avalon-MM slave, requires the CPU to issue an ordinary store instruction to its address. To make the application code simpler, a second custom instruction, `ci_pwm_controller`, was added: it accepts a brightness *level* from 0-9 on `ncs_dataaa`, converts it to the corresponding 8-bit duty value, and issues the Avalon-MM write to `pwm` itself. Because that write can take a variable number of cycles (the peripheral's `avm_waitrequest` signal is not guaranteed to deassert on the very next cycle), this cannot be a single-cycle *Combinational* custom instruction like `count_leading_ones` in Part B; instead it is registered as an *Extended* custom instruction, which adds explicit `ncs_start`/`ncs_done` handshake signals so the Nios II core can stall for exactly as many cycles as the instruction needs. Figure 4.3 shows the resulting two-state controller: `IDLE` waits for `ncs_start` and then latches the converted duty value and asserts `avm_write`; `WRITE` holds that write until `pwm` deasserts `avm_waitrequest`, at which point `ncs_done` pulses for one cycle and control returns to `IDLE`.

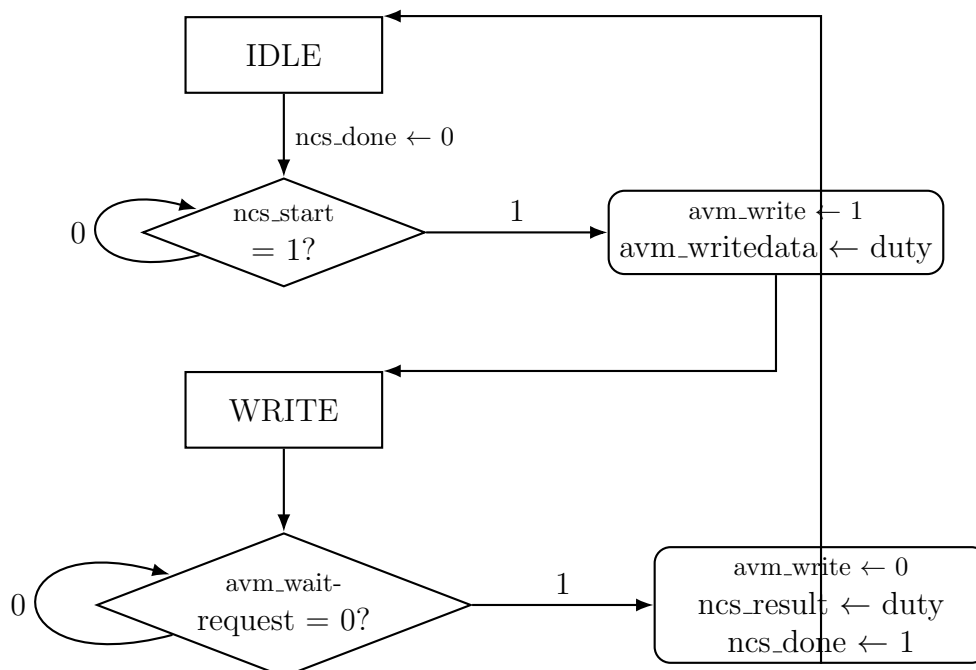


Figure 4.3: ASM-style state diagram for the `ci_pwm_controller` extended custom instruction. Decisions are shown as diamonds and registered actions as boxes, mirroring the Verilog `case` statement below.

```

module ci_pwm_controller #(
    parameter [31:0] PWM_BASE_ADDR = 32'h0008_8000
)(
    // Standalone clock/reset ports, NOT used anywhere in the logic below.
    // These exist purely so Platform Designer's Component Editor
    // auto-detects a genuine Clock Input / Reset Input interface (named
    // "clock" / "reset" once analyzed) for avalon_master_0's Associated
    // Clock / Associated Reset dropdowns to point at - a Custom
    // Instruction Slave's internal clk/reset don't count as a valid
    // clock source for that association. At the system level in
    // Qsys/Platform Designer, wire both of these to clocks_sys... same
    // as every other component; it's the same physical clock as ncs_clk,
    // just exposed a second way so the tool's interface bookkeeping is
    // satisfied.
    input          clk,
    input          reset,

    // Nios II Custom Instruction Slave Interface
    input          ncs_clk,
    input          ncs_clk_en,
    input          ncs_reset,
    input          ncs_start,
    input [31:0]   ncs_dataa,
    output reg [31:0] ncs_result,
    output reg     ncs_done,
    // Avalon-MM Master Interface
    output reg [31:0] avm_address,
    output reg     avm_write,
    output reg     avm_read,
    output reg [31:0] avm_writedata,
    input [31:0]   avm_readdata,
    input          avm_waitrequest
);

    // Brightness level (0-9) -> 8-bit PWM duty value, spanning 0..254
    // in ~28-count steps. Levels 10-15 clamp to full brightness (254)
    // rather than producing an out-of-range or wrapped value.
    function [7:0] level_to_duty;
        input [3:0] level;
        begin
            case (level)
                4'd0: level_to_duty = 8'd0;
                4'd1: level_to_duty = 8'd28;
                4'd2: level_to_duty = 8'd56;
                4'd3: level_to_duty = 8'd85;
                4'd4: level_to_duty = 8'd113;
                4'd5: level_to_duty = 8'd141;
                4'd6: level_to_duty = 8'd170;
            endcase
        end
    endfunction

```

```

        4'd7: level_to_duty = 8'd198;
        4'd8: level_to_duty = 8'd226;
        default: level_to_duty = 8'd254; // level 9, and 10-15 clamp
               here too
    endcase
end
endfunction

localparam IDLE = 1'b0,
            WRITE = 1'b1;

reg state;

always @(posedge ncs_clk or posedge ncs_reset) begin
    if (ncs_reset) begin
        state      <= IDLE;
        ncs_done    <= 1'b0;
        ncs_result  <= 32'd0;
        avm_write   <= 1'b0;
        avm_read    <= 1'b0; // this master never reads
        avm_address <= PWM_BASE_ADDR; // constant, set once
        avm_writedata <= 32'd0;
    end else if (ncs_clk_en) begin
        case (state)
            IDLE: begin
                ncs_done <= 1'b0;
                if (ncs_start) begin
                    avm_writedata <= {24'd0, level_to_duty(ncs_dataaa[3:0])};
                    avm_write    <= 1'b1;
                    state        <= WRITE;
                end
            end
            WRITE: begin
                if (!avm_waitrequest) begin
                    // Slave accepted the write this cycle
                    avm_write <= 1'b0;
                    ncs_result <= {24'd0, level_to_duty(ncs_dataaa[3:0])};
                    ncs_done  <= 1'b1;
                    state     <= IDLE;
                end
                // else: hold avm_write/avm_writedata steady and wait
            end
            default: state <= IDLE;
        endcase
    end
end
end

```

```
endmodule
```

The module above interfaces to the Nios II core as a Custom Instruction Slave on opcode 1, and separately drives `pwm`'s Avalon-MM slave port through its own Avalon-MM master interface, exactly as shown for the two custom instructions together in Table 2.2 and Figure 2.2.

4.4 System Integration and Pin Mapping

`pwm_out` is exported from `pwm` as a single-bit conduit and connected, outside Qsys, directly to the DE2 board's LEDR10 pin, per the LED assignment given to this student in the assignment brief (Table 1: Leading 1's counting, LEDR10 PWM output). Figure 4.4 shows the completed top-level block diagram, extending the Part A diagram (Figure 2.1) with the `pwm_out` export wired to pin PIN_AA13 (LEDR10).

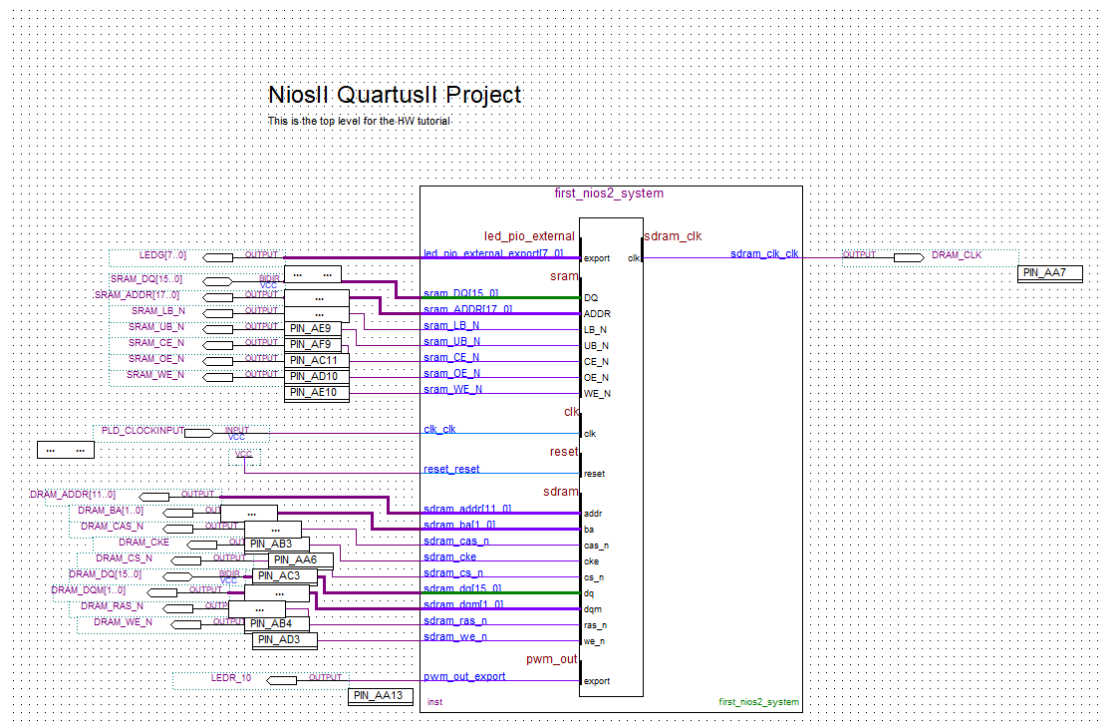


Figure 4.4: Completed top-level block diagram: the Qsys-generated system, its SRAM/SDRAM board pins from Part A, and the `pwm_out` conduit added in Part C, wired to LEDR10 (PIN_AA13).

4.5 Software Design

`count_binary.c` drives the LED pulsing effect entirely in software: an unsigned `level` variable is ramped from `PWM_LEVEL_MIN` (0) up to `PWM_LEVEL_MAX` (10) and back down again, repeating forever, with each step written out through the `ALT_CI_NCI_PWM_CONTROLLER_0(level, 0)` custom instruction macro generated by Qsys and a fixed delay between steps. This gives the simple, human-readable 0-10 brightness scale that `ci_pwm_controller` was designed to accept, rather than requiring the application to compute raw 8-bit duty values itself.

```
/*
*****

*
* Description
* *****
* Pulses the LEDs up and down in brightness using the PWM custom
* instruction component (ALT_CI_NCI_PWM_CONTROLLER_0). The duty-cycle
* "level" ramps from PWM_LEVEL_MIN up to PWM_LEVEL_MAX and back down again,
* repeating forever, which produces a breathing/pulsing LED effect.
*
* Requirements
* *****
* This program requires a PWM controller custom instruction component
* named 'PWM_CONTROLLER_0' to be present in the system (see system.h for
* the generated ALT_CI_NCI_PWM_CONTROLLER_0(A,B) macro).
*
*****
*/

#include "system.h"

#include <stdbool.h>
#include <stdint.h>
#include <unistd.h>

/* Duty-cycle range accepted by the PWM controller. */
#define PWM_LEVEL_MIN 0
#define PWM_LEVEL_MAX 10

/* Delay between duty-cycle steps, in microseconds. */
#define PWM_STEP_DELAY_US 200000

int main()
{
    uint32_t level = PWM_LEVEL_MIN;
    bool increment = true;
```

```

while (true)
{
    ALT_CI_NCI_PWM_CONTROLLER_0(level, 0);

    if (increment)
    {
        if (level == PWM_LEVEL_MAX)
            increment = false;
        else
            level++;
    }
    else
    {
        if (level == PWM_LEVEL_MIN)
            increment = true;
        else
            level--;
    }

    usleep(PWM_STEP_DELAY_US);
}

return 0;
}

```

4.6 Testing and Results

With $\text{PWM_CLK_DIV} = 194$ and $\text{PWM_RESOLUTION} = 254$, one full PWM period is $195 \times 255 = 49,725$ system clock cycles. At the DE2's 50 MHz system clock, this gives a PWM switching frequency of approximately

$$f_{\text{PWM}} = \frac{50 \times 10^6 \text{ Hz}}{49,725} \approx 1.01 \text{ kHz},$$

comfortably above the flicker-fusion threshold of the human eye ($\sim 50\text{--}60$ Hz), so the LED brightness appears steady at each level rather than visibly flickering. Table 4.1 lists the level-to-duty mapping implemented in `ci_pwm_controller`. On the software side, each of the 10 up/down steps is held for $\text{PWM_STEP_DELAY_US} = 200$ ms, giving a full off $\rightarrow$ on $\rightarrow$ off breathing cycle of $2 \times 10 \times 0.2 \text{ s} = 4$ seconds, repeating indefinitely.

Table 4.1: Brightness level to 8-bit PWM duty mapping implemented in `ci_pwm_controller`.

Level	0	1	2	3	4	5	6	7	8	9 (and 10)
Duty	0	28	56	85	113	141	170	198	226	254

The completed system was demonstrated running on the DE2 board; a short video is available at <https://youtube.com/shorts/yChccSzzIE8>, showing LEDR10 fading smoothly from off to full brightness and back to off, repeating continuously, matching the behaviour required by the assignment brief. Part C has no external asynchronous inputs – the brightness level is generated entirely by software and both custom instruction FSMs (`count_leading_ones` and `ci_pwm_controller`) are clocked from `ncs_clk`, the same clock domain as the Nios II core issuing them – so, unlike a design driven by an unsynchronised physical button or switch, there is no metastability or asynchronous-input hazard to account for in this part of the design.

Chapter 5

Conclusions

This assignment met all three learning outcomes it targeted: an ASM-based design method was used for both the PWM controller (Figure 4.2) and the `ci_pwm_controller` custom instruction (Figure 4.3) (LO1); every hardware block – `count_leading_ones`, `pwm`, and `ci_pwm_controller` – was implemented in synthesisable Verilog (LO2); and all of it was integrated into a working Nios II SOPC system in Quartus/Qsys, synthesised for and run on the DE2’s Cyclone II FPGA (LO4).

5.1 What Worked

Part A’s dual-memory verification strategy – swapping the Nios II Processor’s reset/exception vector memory between SRAM and SDRAM so the running test program never overlapped the region under test – worked cleanly and both memories passed all sub-tests (data bus, address bus, byte/half-word access, and per-bit) over their full ranges. Part B’s custom instruction produced a correct, constant-time leading-ones count and measured an $\sim 8\times$ speed-up over the equivalent C routine, with both implementations agreeing on every result checked. Part C’s PWM system produced the required smooth, repeating off–on–off breathing effect on LEDR10, confirmed on real hardware in the demonstration video, at a PWM switching frequency well clear of visible flicker.

5.2 What Didn’t Work Initially

A few issues came up during development and were resolved before final testing. The memory test’s destructive nature meant the first attempt at testing SDRAM corrupted the running program, since the executable itself was still linked into SDRAM; this was fixed by moving the reset/exception vector (and hence the BSP’s default section placement) to the other memory for each test (Figures 2.4 and 2.5).

The standalone PWM ASM design carried over from the previous assignment also needed rework before it could be used here: its ports were renamed and its write path re-timed to be recognised as a proper Avalon-MM slave by Qsys Component Editor, rather than remaining a free-standing block wired up by hand. Finally, an early version of `count_binary.c` checked `level` against its minimum/maximum bounds to flip the ramp direction but never actually incremented or decremented `level` itself, so the custom instruction was always called with `level = 0` and no pulsing occurred; this was caught during testing and fixed.

5.3 Critical Assessment

Part A. Strengths: the design cleanly separates SRAM and SDRAM testing by relocating only the CPU’s reset/exception vector, and both memories are verified over their full advertised capacity. Weakness: this approach requires two separate BSP rebuilds and re-programming cycles rather than a single run that exercises both memories simultaneously under load.

Part B. Strengths: the combinational priority-encoder implementation gives constant, data-independent single-cycle latency and a measured $\sim 8\times$ speed-up over software, using a small, well-structured circuit. Weakness: it consumes FPGA logic resources and one of a limited number of custom instruction opcode slots, and – unlike the software routine – is tied to this specific Nios II system rather than being portable to another target without hardware changes.

Part C. Strengths: splitting the design into a reusable, general-purpose `pwm` Avalon-MM peripheral and a separate `ci_pwm_controller` custom instruction keeps the duty generation logic reusable while giving application code a simple 0–9 brightness scale instead of raw duty values. Weaknesses: the level-to-duty mapping in Table 4.1 is linear, whereas human brightness perception is closer to logarithmic, so the perceived brightness steps are uneven (they appear to change faster at the low end than the high end); and levels 9 and 10 both saturate to the same maximum duty value, making them visually indistinguishable.

5.4 How the System Was Fully Tested

Part A was tested using the Nios II IDE’s Memory Test utility in both directions, exercising the complete 512 KB SRAM and 8 MB SDRAM address ranges (Table 2.3) and all of its built-in sub-tests. Part B was tested with a 100,000-iteration timed benchmark that ran the identical pseudo-varied input pattern through both the software routine and the hardware custom instruction, so their results and timings could be compared directly on equal terms (Table 3.2). Part C was tested

by observing the physical LEDR10 output on the DE2 board over multiple full breathing cycles, recorded in the linked demonstration video, confirming the LED reaches both its off and fully-on extremes and transitions smoothly between them as required.

Bibliography

- [1] Intel Corporation. Intel quartus prime pro edition user guide: Platform designer. <https://cdrdv2-public.intel.com/704996/ug-qpp-platform-designer-19-1-683609-704996.pdf>, 2026. Accessed: 2026-08-04. Platform Designer was formerly named Qsys.
- [2] Wikipedia contributors. Altera. <https://en.wikipedia.org/wiki/Altera>, 2026. Accessed: 2026-08-04.
- [3] Wikipedia contributors. Nios ii. https://en.wikipedia.org/wiki/Nios_II, 2026. Accessed: 2026-07-31.
- [4] Wikipedia contributors. Soft microprocessor. https://en.wikipedia.org/wiki/Soft_microprocessor, 2026. Accessed: 2026-07-31.